RefactorLab API — Sunum Kopya Kâğıdı

Sunucu Uygulama · Kod yolları `src/main/java/com/refactorlab/api/` altındadır · IntelliJ: dosya aç **Ctrl+Shift+N**, satıra git **Ctrl+G**

1. Projeyi 30 saniyede anlat

„Eski, kötü yazılmış **Java kodlarını ve SQL sorgularını** alıp içindeki sorunları (**kod kokusu**) buluyoruz. Her sorun için Martin Fowler kataloğundan **refactoring önerisi** veriyoruz ve kodun hızını **Big-O** ile tahmin ediyoruz. Kodlar, analizler, kurallar ve kullanıcılar **MySQL**'de saklanıyor; kim neyi yapabilir rollerle sınırlı."

Benzetme: Kod hasta, API doktor: hastalığı (kod kokusu) söyler, tedaviyi (refactoring tekniği) yazar. **Örnek:** „Legacy order total" → N+1 Query → öneri JOIN FETCH → O(n²) → O(n).

Akış: Postman → **Controller** (garson: isteği alır) → **Service** (aşçı: kuralları uygular, @Transactional) → **DAO** (depocu: EntityManager ile DB) → **MySQL**

2. Hoca „… nerede?" diye sorarsa

Soru	Ne söylersin	Göster (dosya : satır)
@Lazy nerede?	Katalog uygulama açılırken değil, **ilk analiz isteğinde** oluşur. O anda konsola log düşer.	`config/AppConfig.java:23` `service/AnalysisServiceImpl.java:36`

@Primary / @Qualifier?	Aynı arayüzün iki motoru var: Java (varsayılan, @Primary) ve SQL (ismiyle istenir, @Qualifier).	`common/JavaCodeSmellEngine.java:17`
		`common/SqlQuerySmellEngine.java:15`
		`AnalysisServiceImpl.java:34-35`

@PostConstruct / @PreDestroy?	Katalog oluşunca 16 kaydı yükler, uygulama kapanırken temizler.	`common/RefactoringCatalog.java:25 / :47`
@Configuration + @Bean?	Şifreleyiciyi (Argon2) ve kataloğu kendimiz bean olarak tanımladık.	`config/AppConfig.java:11, 15, 22`
@Value?	Max kod uzunluğu (10000) application.properties'ten, **constructor parametresi** olarak gelir.	`service/CodeSnippetServiceImpl.java:41`
Dependency injection?	Sadece **constructor injection**. Alanda @Autowired yok.	`service/AnalysisServiceImpl.java:31-44`
@RestController?	HTTP isteklerini karşılayan sınıf.	`rest/CodeSnippetRestController.java:28`
@Service?	İş kuralları burada (uzunluk, id kontrolü).	`service/CodeSnippetServiceImpl.java:22`
@Repository + EntityManager?	Spring Data JPA **yok**; sorguları kendimiz yazdık.	`dao/CodeSnippetDAOJpaImpl.java:11`
persist / merge / remove?	Ekleme persist, güncelleme merge, silme remove.	`dao/CodeSnippetDAOJpaImpl.java:24, 52, 60`
Parametreli JPQL?	Dile göre filtre: `?language=SQL`, setParameter ile.	`dao/CodeSnippetDAOJpaImpl.java:45`
@Transactional nerede?	Sadece Service katmanında; DB işlemi ya tamamen olur ya hiç olmaz.	`service/CodeSnippetServiceImpl.java`
POST'ta id gelirse?	Yok sayılır: `setId(0)` → her POST yeni kayıt.	`rest/CodeSnippetRestController.java:60`
PATCH nasıl?	Sadece gönderilen alan değişir (JsonMapper.updateValue). Gövdede id varsa **400**.	`service/CodeSnippetServiceImpl.java:95-102`
Hata yönetimi?	Tek merkez (@RestControllerAdvice) tüm hataları aynı JSON'a çevirir: status, message, timeStamp.	`rest/RefactorLabRestExceptionHandler.java:16`
Güvenlik?	HTTP Basic + SecurityFilterChain; kullanıcı/rol MySQL'de (JdbcUserDetailsManager).	`security/SecurityConfig.java:31`
Şifreler?	**Argon2id** hash'i olarak saklanır, düz metin yok.	`config/AppConfig.java:17`
401 / 403 JSON?	Kimlik hatası ve yetki hatası da JSON gövdeli döner.	`security/RefactorLabSecurityErrorHandler.java`
Actuator / Swagger?	Sağlık + bilgi sayfası; otomatik API dokümanı.	`localhost:8080/actuator/health` `localhost:8080/swagger-ui.html`
Lombok?	Yok, getter/setter elle yazıldı.	`entity/CodeSnippet.java`

3. Hoca „neden / farkı ne?" diye sorarsa

Soru	Kısa cevap
PUT ile PATCH farkı?	PUT kaydın **tamamını** değiştirir (tüm alanlar gönderilir). PATCH sadece **gönderilen alanları** değiştirir.
401 ile 403 farkı?	**401:** Seni tanımıyorum (giriş yok / şifre yanlış). **403:** Seni tanıyorum ama bu işe yetkin yok.
400 ile 404 farkı?	**400:** İstek bozuk (eksik alan, yanlış format). **404:** İstenen kayıt yok.
Neden constructor injection?	Bağımlılıklar zorunlu ve değişmez (final) olur, test etmesi kolaydır, eksik bağımlılık açılışta fark edilir.
Neden @Transactional Service'te?	Bir iş birden çok DB işlemi içerebilir (ör. silerken önce analizler, sonra kod). Hepsi birlikte başarılı ya da birlikte geri alınır.
Neden Spring Data JPA yok?	Ders gereği: EntityManager ile persist/find/merge/remove ve JPQL'i kendimiz yazmamız istendi.
Neden @Lazy?	Katalog her istekte gerekmez; uygulama daha hızlı açılır, ihtiyaç olunca oluşur.

Soru	Kısa cevap
Neden Argon2id?	Modern, yavaş ve bellek isteyen hash algoritması; kaba kuvvet saldırısını zorlaştırır.
Veriler kalıcı mı?	Evet, MySQL'de. Uygulama kapanıp açılsa da durur (ddl-auto=update tabloları silmez).
Neden POST'ta setId(0)?	İstemci id gönderip var olan kaydın üzerine yazmasın diye; her zaman yeni kayıt oluşur.

4. Kullanıcılar (şifre hepsinde: test123)

Kullanıcı	Yetki
viewer	Sadece okur (GET)
developer	Okur + ekler (POST) + günceller (PUT/PATCH) + analiz eder
reviewer	Hepsi + **siler (DELETE)** + kuralları yönetir + actuator

5. Canlı gösterim (Postman → RefactorLab API)

Hoca der ki	Postman'de seç → Send	Beklenen
Listele	1 → Listele - GET	200 + liste
Ekle	1 → Ekle - POST	201
PATCH at	1 → Kısmi güncelle - PATCH	200, sadece başlık değişir
Analiz et	1 → Analiz et - POST	Kokular + öneri + Big-O
Sil	1 → Sil - DELETE (sonra MySQL'de göster)	200
Yetkisiz silme	2 → 403 - DEVELOPER ile silme	403
Olmayan kayıt	2 → 404 - Olmayan kayıt	404
Girişsiz istek	2 → 401 - Kimlik bilgisi yok	401
PATCH'te id	2 → 400 - PATCH gövdesinde id	400
Hepsini çalıştır	RefactorLab API → Run → Run RefactorLab API	31/31 yeşil

6. MySQL'de gösterme ve acil durum

DELETE sonrası (Workbench): `SELECT id, title, language FROM refactorlab.code_snippet;` → silinen id listede yok.

Sorun	Çözüm
Postman: Could not send request	Uygulama kapalı → IntelliJ'de RefactorlabApiApplication (yeşil Run) ya da `mvnw.cmd spring-boot:run`
Açılışta Access denied / bağlantı hatası	MySQL servisi kapalı → Hizmetler → MySQL267 → Başlat
Port 8080 already in use	Uygulama zaten açık; eski pencereyi kullan ya da kapat
İstek 401 dönüyor	İsteğin Authorization sekmesinde kullanıcı/şifre doğru mu?